

Herald



Herald — HelixQA Integration (Operator Guide)

Field	Value
Revision	2
Created	2026-05-30
Last modified	2026-05-30
Status	active
Status summary	r2: closed the api-bank wiring gap. challenges/helixqa-banks/herald-api-v1.yaml was rewritten from prose action: strings (which parse as ActionTypeDescription and never execute, and which helixqa run --platform could never select since it accepts only android/web/desktop/all, not api) to typed shell: curl actions that GENUINELY EXECUTE on the desktop orchestrator path with assertions that BITE (exact-match ["\$code" = "401"], python3 assert on JSON bodies, grep -q on the metrics gauge/event_parser: tag/X-Herald-Replay header). Cases now declare platforms: [api, desktop]. The launcher (scripts/helixqa_run.sh) exports HELIXQA_HTTP_BASE_URL when the live serve is up and runs --platform desktop (not all, which added phantom UI-platform skips). Self-validated against a self-signed-HTTPS stub: 11/11 PASS (good responses) vs 10/11 FAIL (wrong responses) — bite proof under qa-results/api-bank-rewrite-validation/. HTTP-executor TLS-verify limitation documented (no InsecureSkipVerify → http: action unusable for the self-signed listener → shell: + curl -k is the fit). Prior r1: initial operator-facing guide.
Issues	none
Issues summary	—
Fixed	api-bank wiring gap closed — prose action: strings → typed shell: curl actions that genuinely execute with biting assertions (r2)

Field	Value
Continuation	bump when the api bank gains coverage of new pherald /v1/* routes (e.g. a future /v1/subscribers surface); bump when additional flavors ship serving subcommands worth an HTTPS probe; bump when HELIXQA_AUTONOMOUS=1 becomes the default after the autonomous session is validated against a live claude + PG environment; bump when the release gate (scripts/release.sh) wires helixqa_run.sh in as a tag-time blocker.

Table of contents

- [What HelixQA is](#)
- [Sibling layout \(CONST-051\)](#)
- [The LLM / Vision bridge](#)
- [Running the QA](#)
- [The test banks](#)
- [Evidence layout](#)
- [Where HelixQA fits — the autonomous-QA SSoT](#)
- [Troubleshooting](#)

What HelixQA is

[HelixQA](#) is an **anti-bluff QA orchestration framework** for cross-platform testing with real-time crash detection, step validation, evidence collection, and automated ticket generation. Its design centre is the Helix §11.4 Operative Rule — the same covenant Herald restates at §107: **the bar for shipping is not "tests pass" but "the end user can use the feature."** Every PASS HelixQA emits must carry positive runtime evidence captured during execution; a green summary line without that evidence is a critical defect.

HelixQA consumes **YAML test banks** that describe platform-targeted test cases (structure, not prose) and — in autonomous mode — drives an LLM-powered session that navigates the system under test, verifies documented features, hunts for bugs, and writes a QA report with evidence.

For Herald, HelixQA validates two surfaces:

1. **The pherald HTTP API** (platforms: [api, desktop]) — the live Gin /v1/* plane: the JWT-gated POST /v1/events ingestion route plus the healthz / readyz / metrics built-ins. The cases declare BOTH api (the semantic surface tag) and desktop (the platform whose orchestrator path genuinely executes their shell: curl actions — see the "API-bank execution model" note below).
2. **The eight flavor binaries** (platforms: [desktop]) — pherald, sherald, cherald, bherald, rherald, iherald, scherald, qaherald — each exercised end-to-end as a compiled binary. This is the §107 "the binary actually runs for the end user" check that Herald has repeatedly regressed on (unit tests call the cobra constructors and bypass main(), so eager-dependency and flag-ordering bugs hide there).

Sibling layout (CONST-051)

HelixQA lives as a **sibling repository**, not a Herald submodule. Per CONST-051 (submodules-as-equal-codebase + flat-layout, no nested own-org submodule chains), the expected on-disk arrangement is:

```
~/Projects/
├── Herald/                ← this repo
│   ├── challenges/helixqa-banks/ ← Herald's banks (loaded by helixqa)
│   └── helixqa/           ← the HelixQA framework
│       ├── bin/helixqa    ← built by scripts/helixqa_run.sh if missing
│       └── Challenges/    ← digital.vasic.challenges (HelixQA dependency)
└── Containers/           ← digital.vasic.containers (HelixQA dependency)
```

The launcher resolves HelixQA at `~/Projects/helixqa` by default; override with the `HELIXQA_DIR` environment variable. HelixQA's autonomous session hard-codes its bank-discovery path to `<project>/challenges/helixqa-banks` — which is exactly where Herald's banks live.

The LLM / Vision bridge

HelixQA's autonomous session and its prose-step bank evaluation are LLM-driven. It discovers providers two ways:

- **API-key providers** — `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OPENROUTER_API_KEY`, `GROQ_API_KEY`, ... (40+ supported via the registry in `pkg/llm`).
- **Bridged CLI providers** — when a CLI tool such as `claude`, `qwen-coder`, or `opencode` is on `PATH`, HelixQA wraps it in a `BridgedCLIProvider` (zero API-key cost). The `claude` bridge is also **vision-capable**, so it doubles as the screenshot/visual analyzer.

Anti-bluff hard gate. `scripts/helixqa_run.sh` **FAILS LOUD** if `claude` is **not** on `PATH`. Rationale: with no provider at all, the autonomous session degrades to an observation-only mode that can print a green line *without* doing any real anti-bluff verification — exactly the "absence-of-error PASS" the §107 covenant forbids. Refusing to start is the honest behaviour. Install Claude Code and confirm command `-v claude` resolves before running.

Running the QA

```
# From the Herald repo root:
scripts/helixqa_run.sh
```

What the launcher does, in order:

1. **Preflight** — fail loud if `claude` is not on `PATH`; verify both banks exist.
2. **Build helixqa** — if `~/Projects/helixqa/bin/helixqa` is missing, go `build ./cmd/helixqa` (build log captured).
3. **Build all 8 flavor binaries** into the run's `bin/` directory so the CLI bank exercises freshly-compiled binaries.
4. **Boot a real pherald** serve for the API bank — mirroring `scripts/e2e_bluff_hunt.sh` (E7–E12): when Postgres is reachable on `:24100`, start `pherald serve` with `HERALD_PG_DSN+HERALD_AUTH_MODE= hmac + HERALD_AUTH_HMAC_SECRET`, wait for the **HTTPS** listener (Wave 4a — probe with `curl -k`), and capture a live `healthz` body as evidence. On success the launcher **exports** `HELIXQA_HTTP_BASE_URL=https://127.0.0.1:$ {HERALD_HTTP_PORT}` so the API bank's `shell: curl` steps target the real listener. When PG is unreachable, the live serve is **SKIPPED with a recorded reason** (§11.4.3)

and the API bank is left out of the run so no API coverage is claimed that was not exercised.

5. **Run the banks** — `helixqa run --banks <api,cli> --platform desktop` writing report + evidence under `qa-results/helixqa/<run-id>/.desktop` is the platform whose orchestrator path genuinely executes `shell: actions` (both banks declare only desktop-executable steps); `--platform all` would add phantom SKIP rows on the `android / androidtv / web` platforms and mask the real desktop verdicts.
6. **(Optional) autonomous session** — set `HELIXQA_AUTONOMOUS=1` to also run `helixqa autonomous --project <repo> --platforms api,desktop`.
7. **Graceful teardown** — the `pherald serve` process is `SIGTERM`'d on exit. Any FAIL makes the script exit non-zero (release-gate composable).

Environment overrides: `HELIXQA_DIR`, `HERALD_HTTP_PORT` (default 24791), `HERALD_PG_DSN`, `HERALD_PG_PORT` (default 24100), `HERALD_AUTH_MODE`, `HERALD_AUTH_HMAC_SECRET`.

The test banks

Banks live under `challenges/helixqa-banks/` and follow the HelixQA YAML schema: `top-level version / name / test_cases[]`; each case carries `id / name / category / priority / platforms[] / steps[]` (each step `name / action / expected`) / `tags[] / documentation_refs[]`.

Bank	Platforms	Cases	Coverage
herald-api-v1.yaml	api, desktop	11	Real pherald /v1/* routes via typed <code>shell: curl</code> actions: <code>healthz</code> , <code>readyz</code> , <code>metrics</code> , and <code>POST /v1/events</code> (no-token 401, forged-token 401, valid-token 202 + Receipt, idempotency replay 200 + X-Herald-Replay, malformed body 400 <code>event_parser:</code> , missing-tenant-claim 401, 404 boundary, /v1/compliance not-served-by-pherald).
herald-cli-flavors.yaml	desktop	10	All 8 flavor binaries' <code>version --json</code> canonical shape + human <code>DisplayName</code> line; <code>pherald --help</code> subcommand discoverability; unknown-subcommand fail-loud. Self-asserting <code>shell: actions</code> (see below).

CLI-bank execution model — `shell: actions` = real execution (not crash-absence)

Every `herald-cli-flavors.yaml` step uses `action: "shell: <cmd>".` HelixQA's desktop challenge path (`executeDesktopShellSteps`) runs each via `sh -c`, captures the real exit code + combined output, and scores PASS only when the step exits 0. The command itself encodes the assertion — it pipes the binary's stdout through `grep` ("`binary`": "`pherald`", the flavor code, the `DisplayName`, the named subcommands), so a wrong field or branding regression forces a non-zero exit → FAIL. A prose (non-`shell:`) action would instead SKIP honestly with a `"convert to action: \"shell: <cmd>\""` reason — and the aggregator will NEVER promote a desktop skip to PASSED on crash-absence (desktop has no persistent app, so crash-absence is not CLI evidence; the `promoteSkippedToPassed` desktop guard in `helixqa` enforces this). Net: a desktop PASS is earned only by genuine execution, and the captured command + exit code is rendered into `qa-report.md` under "### Recorded evidence" as the auditable §107.x proof. Mutation-proven: corrupting one case's

assertion flips it PASSED→FAILED. See docs/qa/HRD-QA-CLI-FLAVORS-<run-id>/ for a captured run.

API-bank execution model (why shell: curl, not http:)

The original API bank declared platforms: [api] with prose action: "GET https://... (curl -k)" strings. Those were a **known wiring gap**: the HelixQA run orchestrator only EXECUTES shell: action steps, and only on the desktop platform path (pkg/orchestrator/definition_challenge.go → executeDesktopShellSteps, os/exec, PASS only when every step exits 0). Prose strings parse as ActionTypeDescription and never run — and helixqa run --platform itself only accepts android|web|desktop|all (no api), so an api-only case was never even selected. The cases looked covered but exercised nothing.

The gap is now **closed**. Each case uses a typed action: "shell: <curl ... assertion>" step that genuinely runs:

- **HTTPS / self-signed cert** — curl -k (the executor has no TLS skip-verify knob; see the limitation below). HelixQA's http: action type / HTTPExecutor is therefore **not usable** for Herald's listener: it (a) only runs in the *autonomous* StructuredTestExecutor, not the banks-driven run path the launcher uses, and (b) builds a default *http.Client with **no** InsecureSkipVerify, so it cannot reach a self-signed-HTTPS endpoint. shell: + curl -k is the correct fit.
- **Status assertions bite** — curl -k -sS -o /dev/null -w '%{http_code}' captured into a variable, then an exact-match ["\$code" = "401"]. A wrong status exits non-zero → the shell step exits non-zero → the case FAILS.
- **Body / header assertions bite** — python3 -c '...assert...' on the JSON body (health/ready/receipt) and grep -q for the pherald_build_info gauge line, the event_parser: error tag, and the X-Herald-Replay: true response header (captured via curl -D -).
- **JWT-gated cases** mint a real HS256 token inline with python3 (hmac/hashlib/base64), matching scripts/e2e_bluff_hunt.sh's proven minting pattern, signed with HERALD_AUTH_HMAC_SECRET. Case 009 mints a signature-valid token that **omits** the tenant claim to drive the 401 path.

The cases keep api in platforms[] as the semantic surface tag and add desktop so the orchestrator actually executes them. The base URL comes from HELIXQA_HTTP_BASE_URL (exported by the launcher when the live serve is up), falling back to https://127.0.0.1:\${HERALD_HTTP_PORT}.

HTTP-executor limitation (TLS verify). HelixQA's HTTPExecutor (pkg/autonomous/http_executor.go) uses the Go default transport with no InsecureSkipVerify option. Herald's pherald serve listener is self-signed HTTPS (Wave 4a), so the http: action type cannot reach it. This is the documented reason the API bank uses shell: + curl -k rather than http:. If HelixQA later adds a skip-verify knob (e.g. an HELIXQA_HTTP_INSECURE_SKIPVERIFY env or a per-bank flag) AND wires the http: type into the run path, the bank can migrate to http: actions with expect_status / expect_body_contains / expect_json_path fields.

Anti-bluff anchor. Every case asserts on the actual response **body** or **stdout** — status:ok + build.version, status:ready, the pherald_build_info{ gauge line, the event_parser: -tagged error string, the X-Herald-Replay header, binary == "<flavor>" in version --json, the DisplayName on the first stdout line — never a bare

2xx-or-no-error check. The bank expected prose drives the LLM-generated verification prompts at runtime (CONST-046): banks describe **what correct looks like**, not a hard-coded English string match.

Evidence layout

Each run writes a timestamped directory:

```
qa-results/helixqa/<run-id>/
├── bin/                  freshly-built flavor binaries used by the CLI b
├── helixqa_build.log     helixqa go-build log (only if it had to be buil
├── flavor_build.log      go-build log for the 8 flavors
├── pherald_serve.log     stdout/stderr of the live pherald serve (api ba
├── healthz_evidence.json captured live /v1/healthz body (positive eviden
├── api_serve_skip.txt    present only when PG was unreachable (SKIP reas
├── helixqa_run.log       the helixqa run transcript
├── helixqa_autonomous.log present only with HELIXQA_AUTONOMOUS=1
└── qa-report.{md,html,json} + tickets/  HelixQA report + any generated t
```

This satisfies Herald §107.x (the docs/qa/ / qa-results/ evidence mandate): the run-id directory is the auditable runtime proof that the banks were exercised against real services and real binaries.

Where HelixQA fits — the autonomous-QA SSoT

HelixQA is Herald's **autonomous-QA single-source-of-truth**. It is complementary to, not a replacement for, the existing seams:

- `scripts/e2e_bluff_hunt.sh` — the canonical hand-rolled end-to-end smoke (14+ invariants against real services). HelixQA's launcher deliberately mirrors its PG + Gin boot so the two agree on environment.
- `tests/test_*_mutation_meta.sh` — the paired §1.1 mutation gates.
- **HelixQA** — the LLM-driven autonomous layer: it can grow coverage by exploration (curiosity phase), generate tickets for failures, and is the intended home for cross-flavor / cross-platform QA as Herald adds serving flavors and messenger channels (Wave 7+).

The intended end state is for `helixqa_run.sh` to be a release-gate blocker alongside the e2e smoke — a tag cannot ship if the banks FAIL.

Troubleshooting

Symptom	Cause / fix
FAIL: `claude` is not on PATH	Install Claude Code; confirm command <code>-v claude</code> . This is intentional — see the bridge section .
HelixQA sibling repo not found at ...	Clone HelixQA to <code>~/Projects/helixqa</code> (CONST-051 flat layout) or set <code>HELIXQA_DIR</code> .
SKIP api-serve: Postgres :24100 unreachable	Start the quickstart Postgres (<code>podman-compose -f quickstart/docker-compose.quickstart.yml up -d postgres</code>) or point <code>HERALD_PG_DSN</code> / <code>HERALD_PG_PORT</code> at a reachable instance. The CLI bank still runs.

Symptom	Cause / fix
pherald serve never accepted HTTPS within 10s	Inspect <code>qa-results/helixqa/<run-id>/pherald_serve.log</code> — usually a JWT-verifier or DSN misconfig. The listener is HTTPS; probe with <code>curl -k</code> .
go build failed for flavor ...	Inspect <code>flavor_build.log</code> . A fresh clone needs <code>go work init && go work use ./...</code> (go.work is gitignored per spec §9.1).

Sources verified

Per Helix §11.4.99, the external-tool behaviour documented here was cross-referenced against the in-tree HelixQA sources at the time of writing (no external network docs were required — HelixQA is a sibling repo on disk):

- `~/Projects/helixqa/README.md` — bank format, autonomous session, BridgedCLIProvider, CLI subcommands. Verified 2026-05-30.
- `~/Projects/helixqa/cmd/helixqa/main.go` — `run / autonomous` flags, the `<project>/challenges/helixqa-banks` discovery path, the `claude/qwen-coder/opencode` bridge discovery loop. Verified 2026-05-30.
- `~/Projects/helixqa/pkg/config/config.go` — the `api / desktop` platform constants. Verified 2026-05-30.